PalmSource Developer Conference 2004
FEBRUARY 10-12, 2004 SAN JOSE, CA



Palm OS® Cobalt Details: Communications and IOS

Discover the new communications architecture in Palm OS® Cobalt.

Presented By:

Robert Whiteman

DTS Engineer - Comms

palm POWERED

The contents of and the technologies described in this presentation are provided to you and described "AS IS," without any warranty or representation of any kind. PalmSource hereby disclaims each and every other representation or warranty with respect thereto, including the implied warranties of merchantability, fitness for a particular purpose, accuracy and non-infringement. No representation or warranties are made that any product based in whole or in part on any specifications or information referenced herein will be free from defects or safe for use for its intended purposes.

PalmSource further reserves the right to modify in any respect any technology described or content provided herein without prior written notice.

Palm OS is a registered trademark and Palm Powered is a trademark of Palm Trademark Holding Company, LLC. Other brands may be trademarks of their respective owners. All other trademarks and tradenames used herein are the property of their respective owners.

Copyright (c) 2004 PalmSource, Inc.



Introduction

All new architecture

Open standards

Flexible and extensible layered architecture

Improved user experience

Backwards compatibility



Topics

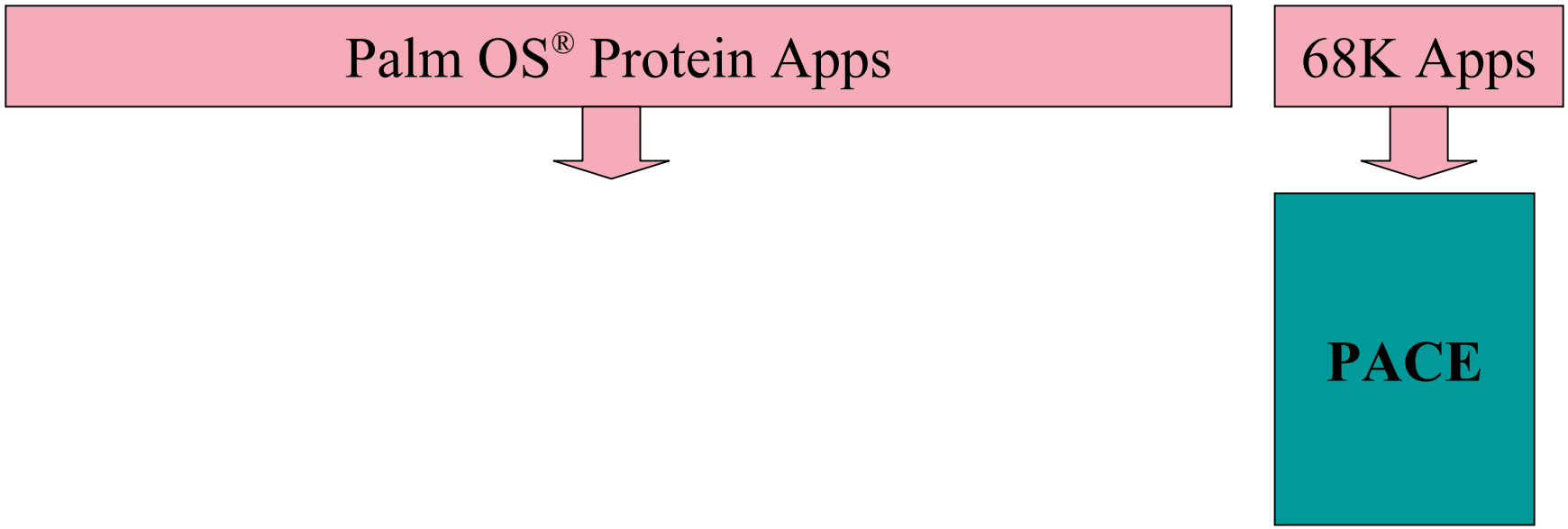
- Communications components
- Which API to use?
- Application designs
- Specific technologies
- Connection Manager
- Q&A



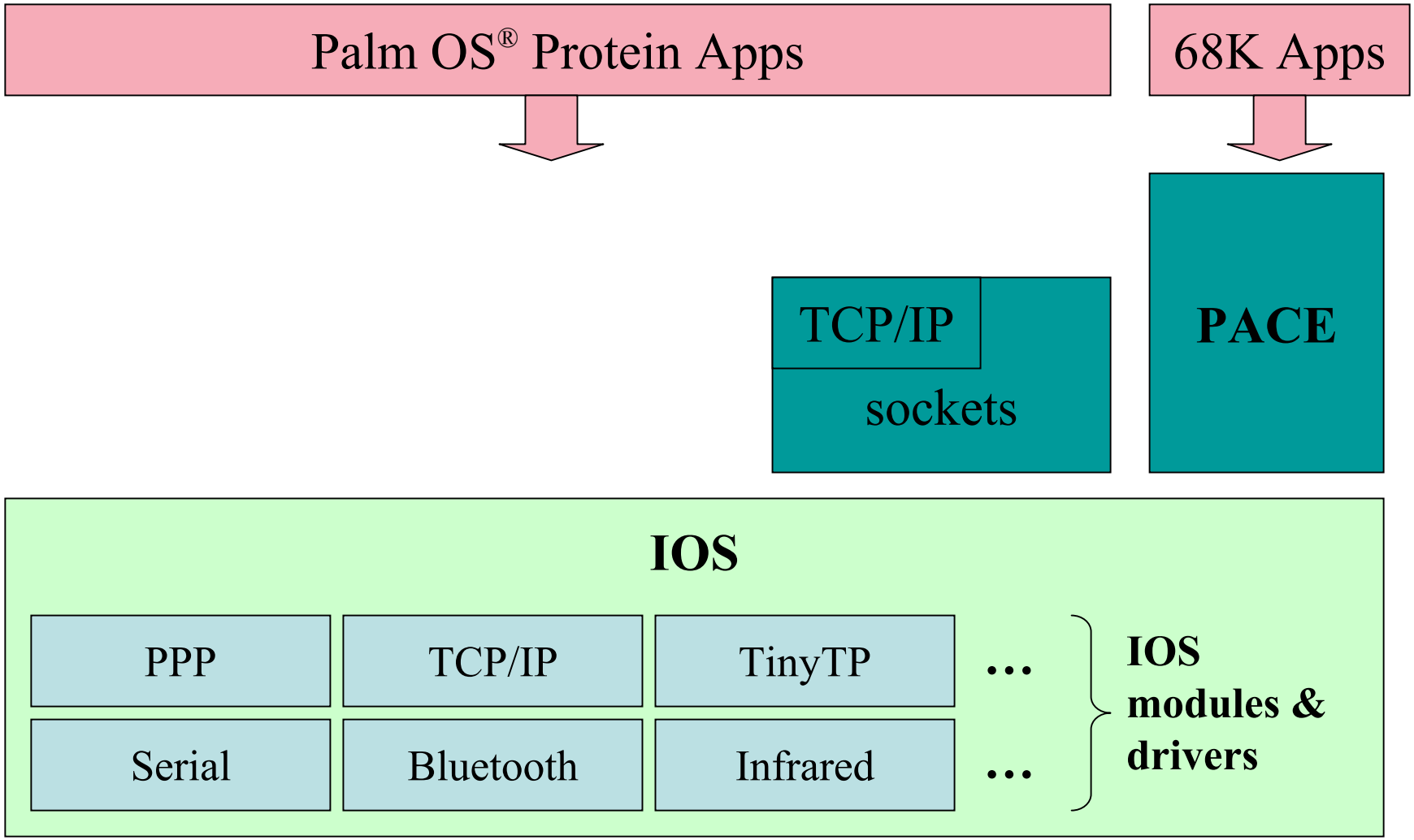
Topics

- **Communications components**
- Which API to use?
- Application design approaches
- Specific technologies
- Connection Manager
- Q&A

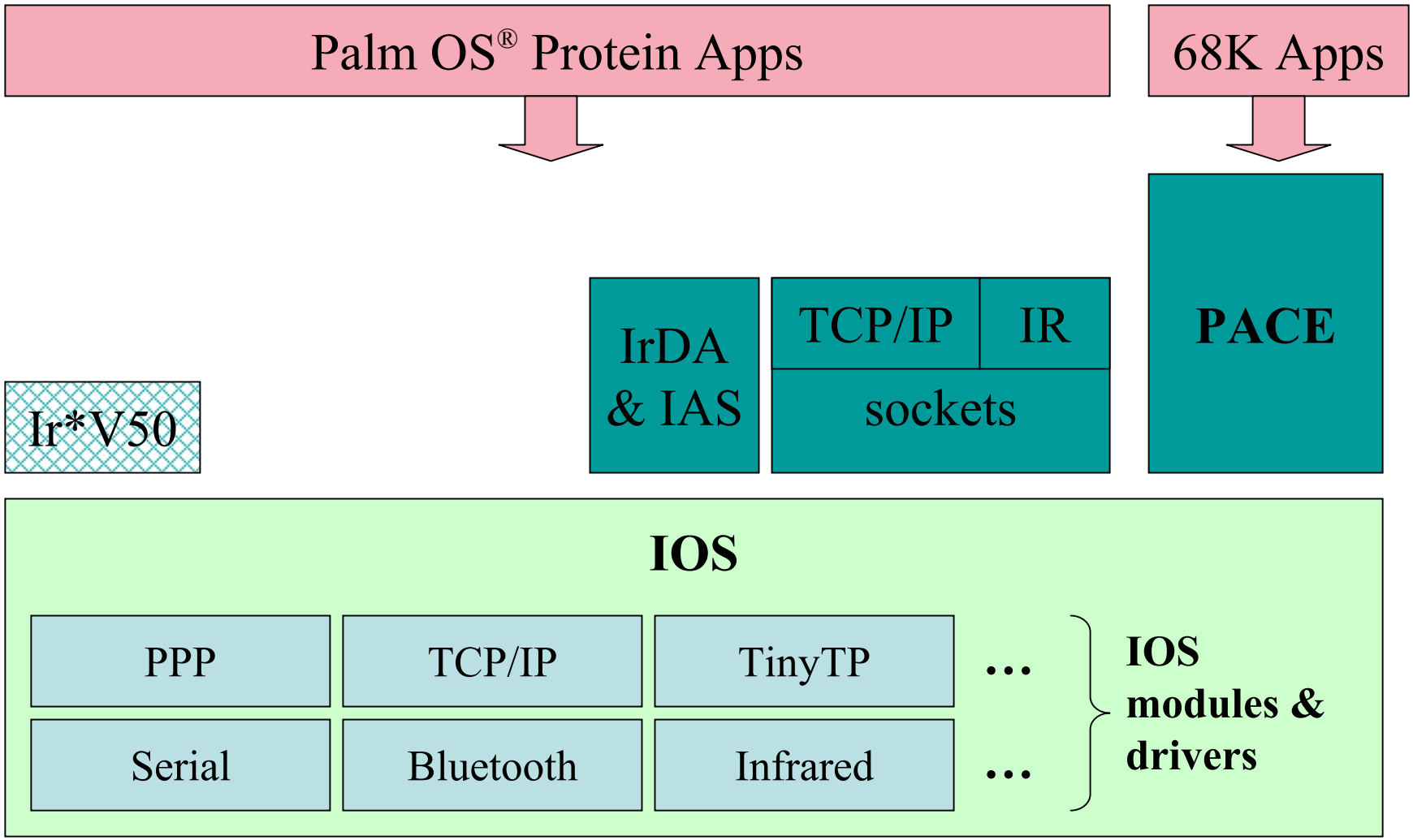
Communications Architecture



Communications Architecture



Communications Architecture





Communications Architecture

Palm OS® Protein Apps

68K Apps

PACE

IrDA & IAS

TCP/IP | IR

sockets

Ir*V50

Srm

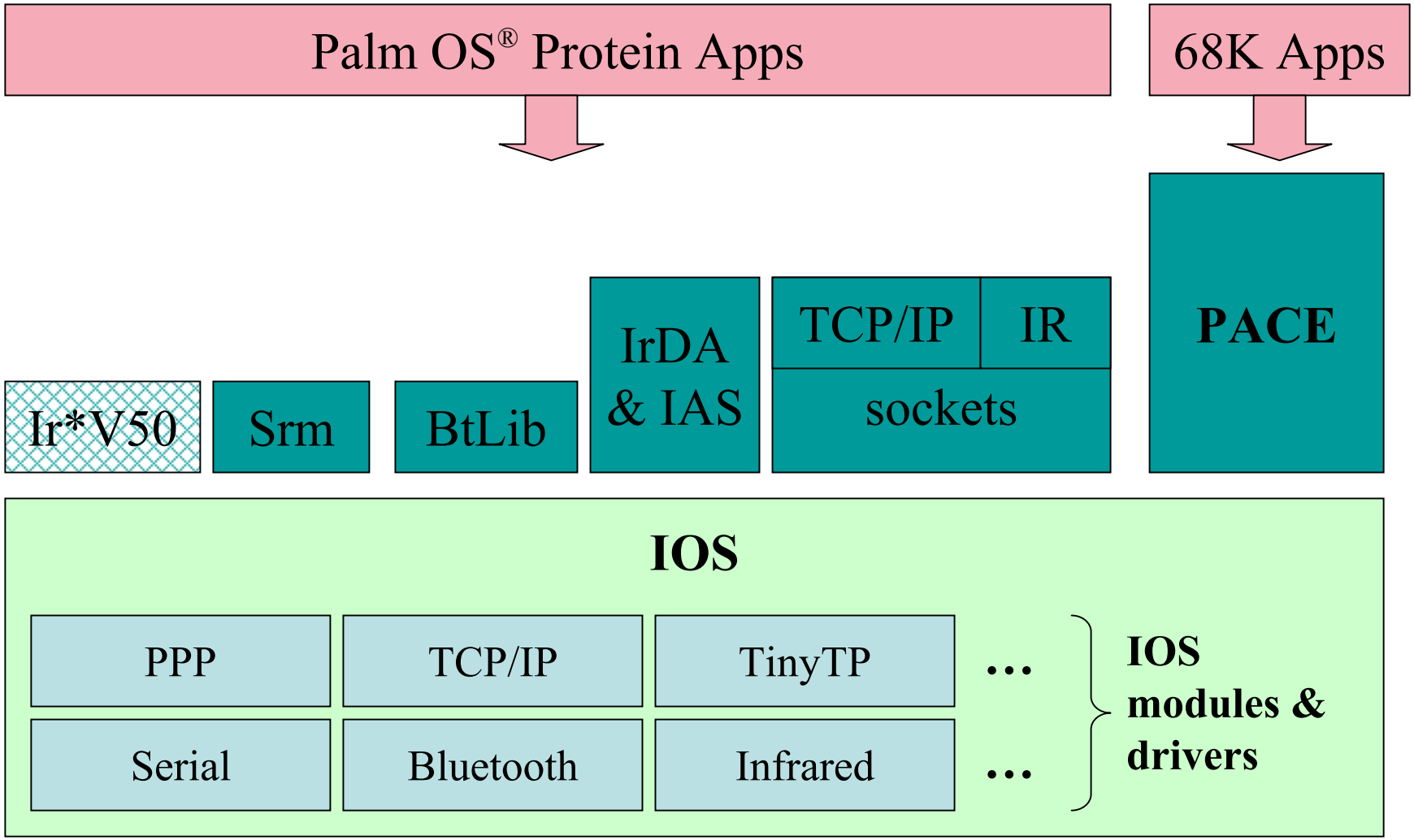
IOS

PPP | TCP/IP | TinyTP | …

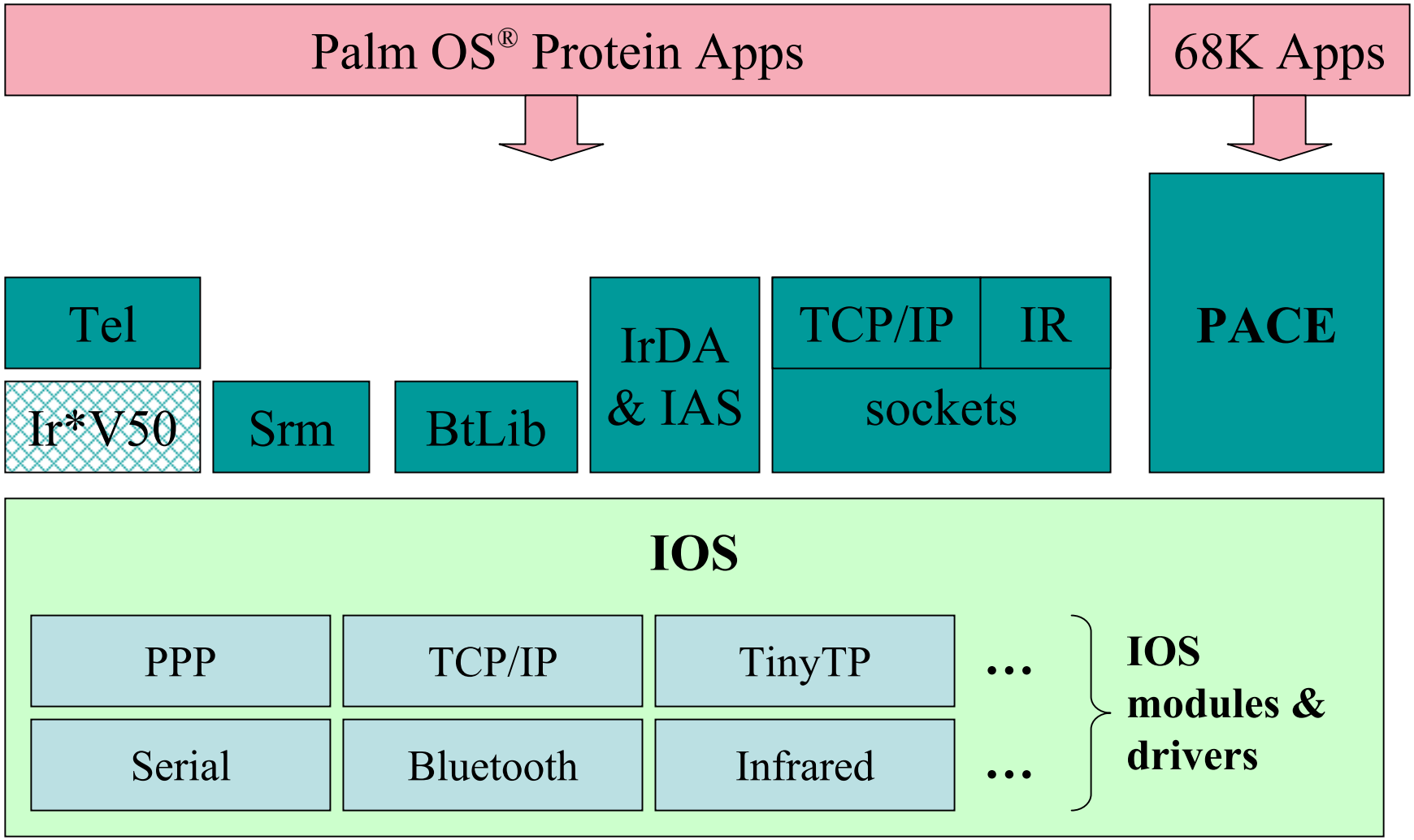
Serial | Bluetooth | Infrared | …

IOS modules & drivers

Communications Architecture

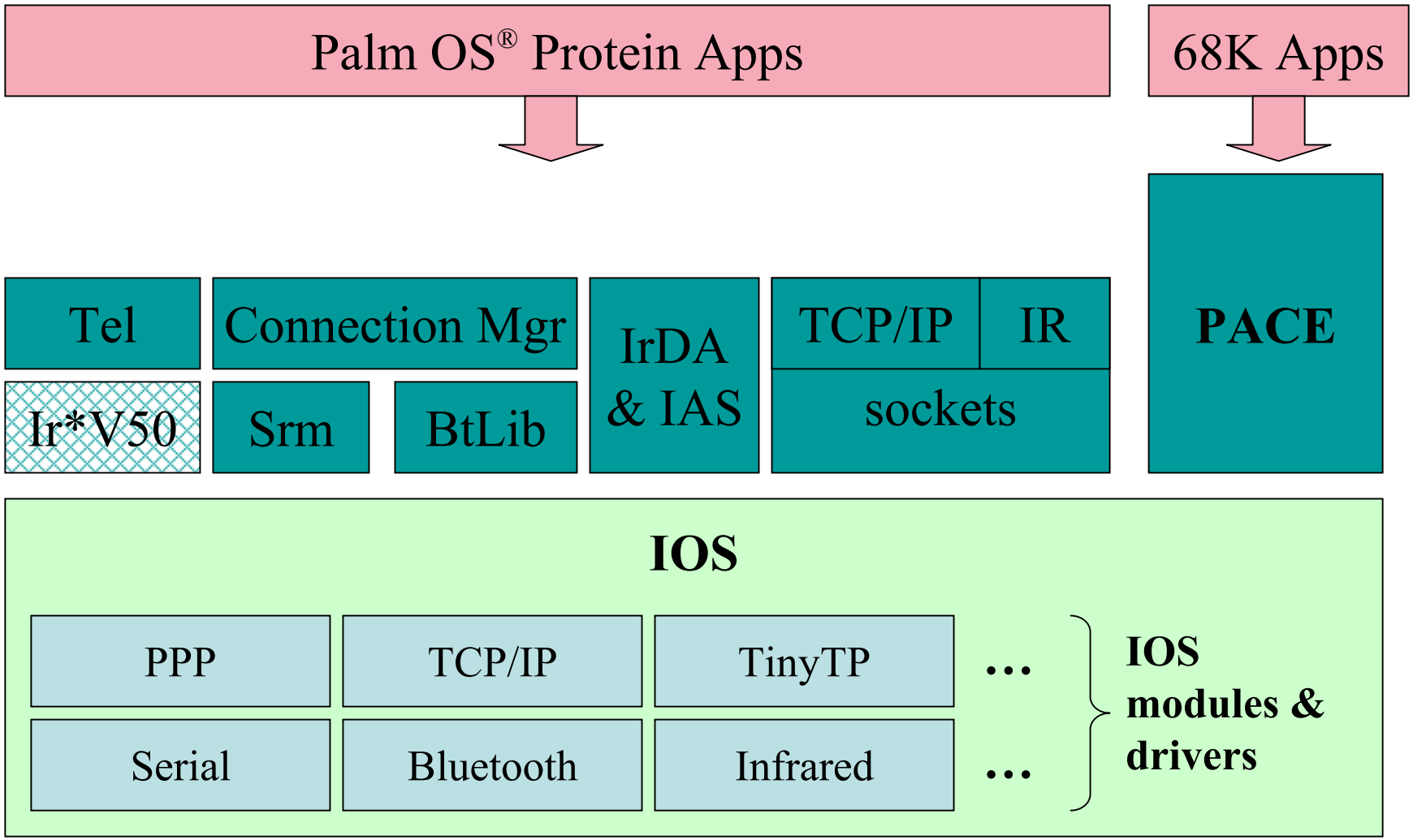


Communications Architecture





Communications Architecture





Communications Architecture

Palm OS® Protein Apps

68K Apps

Exchange Mgr

Tel

Connection Mgr

IrDA & IAS

TCP/IP

IR

sockets

Ir*V50

Srm

BtLib

PACE

IOS

PPP

TCP/IP

TinyTP

…

Serial

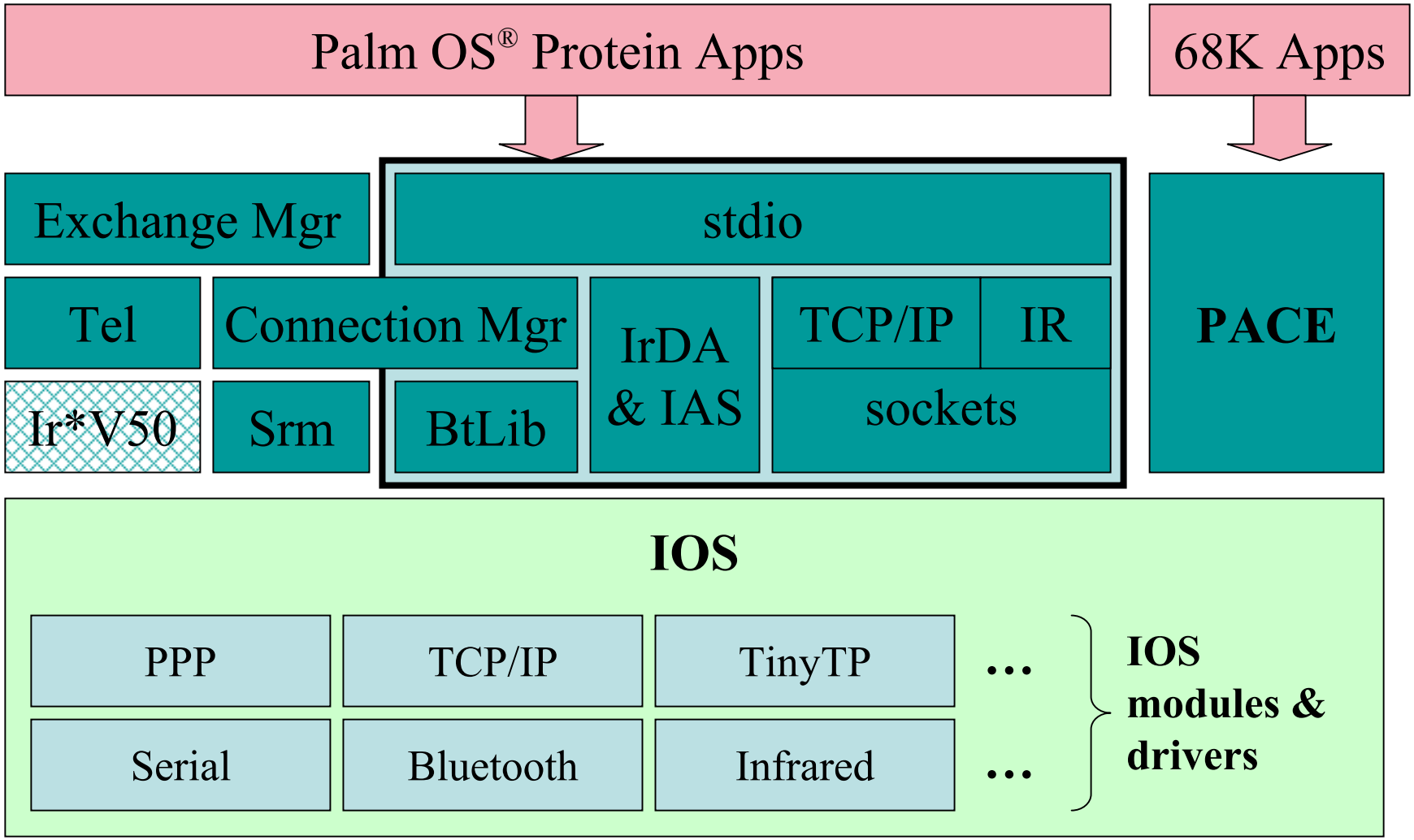
Bluetooth

Infrared

…

IOS modules & drivers

Communications Architecture





Summary

Open standard APIs as well as familiar APIs

Compatibility with old applications

Updated functionality

Entirely new Connection Manager API



Topics

- Communications components
- **Which API to use?**
- Application design approaches
- Specific technologies
- Connection Manager
- Q&A



Compatibility

Well-behaved 68K applications *should* just work without code changes

Many inherent limitations are gone

- No more limit on the number of sockets
- No more virtual serial driver limitations

Some things to watch out for

- Ticks are now milliseconds
- Deprecated and unsupported APIs will no longer work!



Palm OS® Protein API Choices

1. **For simple object exchange, use Exchange Manager**
2. **TCP/IP applications use sockets**
3. **Applications initially ported from 68K use compatibility APIs**

   Consider porting to a better API ASAP
4. **Infrared apps use sockets with AF_IRDA**
5. **Bluetooth apps use BtLib**
6. **Serial apps use Serial Manager**

   Thrill-seekers who need low-level serial line access or want to use stdio functions can use the IOS API with termios.



Topics

- Communications components
- Which API to use?
- **Application design approaches**
- Specific technologies
- Connection Manager
- Q&A



App Design Approaches

Single-threaded and blocking

Polled using IOSPoll

Polled using PollBox

Multi-threaded client

Multi-threaded server



Single-threaded and Blocking

Calls to IOS and stdio functions are blocking

The UI will freeze until the call returns

Some calls may never return!

If the UI task is blocked for too long the OS will ask the user to kill the application

Do not use this approach!



Polled using IOSPoll

Poll for communication activity using `IOSPoll()`

Roughly equivalent to NetLib style communications

Difficult to poll multiple file descriptors

You probably shouldn't use this technique

Use the next one instead



Polled using PollBox

PollBoxes do all of the work for you

Manage multiple connections more easily

Poll for incoming UI events as well

- See `EvtGetEventDescriptor()`

Simple callback mechanism

Don't spend too long handling a callback or the UI will freeze

This is a good approach for client apps

See the documentation in "Low Level Communications"



Multi-threaded Client

- Use threads to handle communications
- Primary thread continues to process UI events
- Create a thread that handles all communications
- The communications code can block without stalling the application
- The application is always responsive to the user
- The drawback is thread synchronization code
- This is the best client approach



Multi-threaded Server

Use threads to handle communications

Primary thread continues to process UI events

Create a thread that blocks waiting for incoming connections

For each incoming connection, create a thread to service it

The application is always responsive to the user

The drawback is thread synchronization code

This is the best server approach

See SimpleServer sample application in the Developer KB



Topics

- Communications components
- Which API to use?
- Application design approaches
- **Specific technologies**
- Connection Manager
- Q&A



Communications Technologies

IOS

TCP/IP

Serial

Infrared

Telephony

Bluetooth



IOS

IOS functions are similar to stdio

`IOSOpen()`	`IOSClose`	`IOSIoctl()`
`IOSRead()`	`IOSWrite()`	`IOSPoll()`
`IOSReadv()`	`IOSWritev()`	PollBoxes
`IOSGetmsg()`	`IOSPutmsg()`	and more…

IOS operates on *file descriptors* (FDs)

There are FDs that represent various concepts

TCP link	Direct device communication
UI Events	Bluetooth Management Events

You won't be able to use IOSWrite() with all FDs

Maximum of 64 FDs per process



TCP/IP

Sockets!

Much higher limit on the number of sockets (~64)

True background and server applications

NetLib exists in PACE for 68K apps, but it is not available for Protein apps



Serial

Most programs will use the familiar `Srm` functions

- Don't mix `IOS` or `stdio` functions with the `Srm` functions

Thrill-seekers may use the IOS API directly

- Open the serial device using `IOSOpen()`
- Use termios calls to query and control the port directly
- Not all termios ioctls are supported
- Use `fdopen()` to convert the file descriptor to a `FILE *` and use the stdio functions



Infrared

The deprecated IRLib API is available only to ease porting

- Functions renamed: `IrFoo()` is now `IrFooV50()`

If you're using IrComm (serial over Infrared) you can still use the serial manager

- Open the `sysFileCVirtIrComm` serial port

The supported API is sockets

- Use the `sockaddr_irda` socket type
- Use `IrDADiscoverDevices()` to get the remote device address
- Use `IAS` functions for more sophisticated service discovery

You can use stdio functions to read and write

- Use `fdreopen()` to get a `FILE *` structure
- Then you can use `fprintf()` `fputc()` and so on.



Telephony

Support for more cellular networks

Functions will be familiar, but not identical

- `TelSpcPlayDTMF()` **is now** `TelSpcPlayTone()`
- `TelSndMute()` is now `TelSndSetMuteStatus()`

Telephony refnum is now a `telDescNum`

New capabilities:

- Loudspeaker volume
- Vibrate mode
- Caller ID settings
- Alert sound mode
- Conference calling
- Ringer volume
- Call forwarding
- Voicemail settings
- Network access
- Call history

New SMS capabilities

- NarrowBand Sockets



Bluetooth

Same profiles as before

BtLib API works the same, mostly

- Management event callbacks are handled using a management file descriptor

Uses IOS APIs

No Virtual Serial Driver

No interference with other systems

Updated built-in Discovery UI

Bluetooth services



Bluetooth Services

Register

`BtLibServiceRegistration()`

Prepare a service record

Application receives `sysBtLaunchCmdPrepareService` notification

Execute the service

Application receives the `sysBtLaunchCmdExecuteService` notification



Topics

- Communications components
- Which API to use?
- Application design approaches
- Specific technologies
- **Connection Manager**
- Q&A



Connection Manager Overview

Configure a new connection

Store a connection for later use

Connect and disconnect

Query connection status

Provide optional configuration UI



What's new about the Connection Manager?

Multiple simultaneous connections

Greatly improved user experience

- One app, instead of multiple Prefs panels
- No inappropriate parameters or unused settings

Extensible plug-in architecture

Applications can have complete control

- Applications can override UI or use default dialogs to prompt users

A single API for all connection types

See the NetSample example in the Developer KB



Connection Manager Components

Connection Manager Core

- Provides the `Cnc*` API

Connection Manager Plug-Ins

- Implement one or more comm layers
- Provide UI

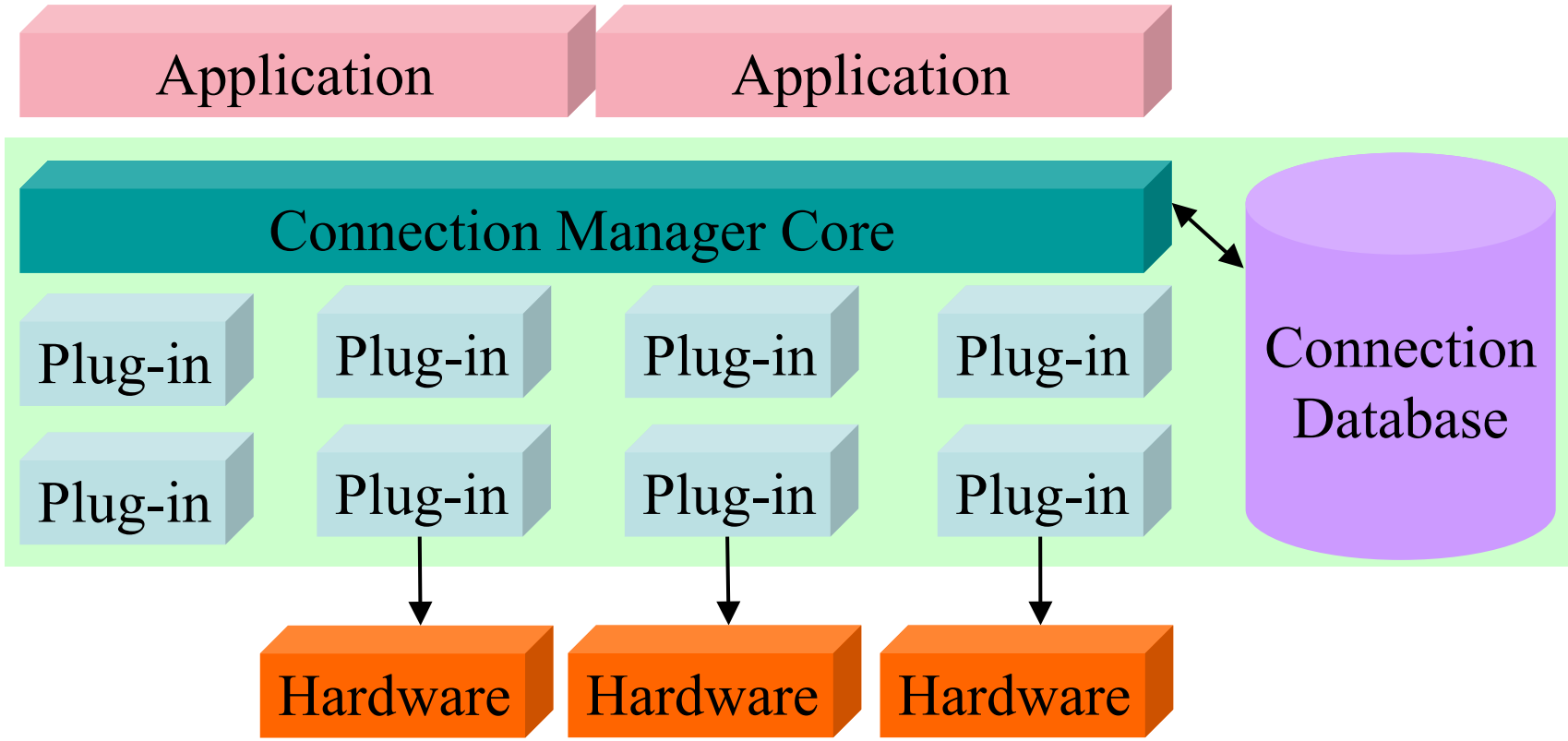
Connection Profiles

- Specify a set of plug-ins and their configuration
- Have a text representation

Connection Database

- Stores profiles and plug-in data

Connection Manager Architecture





Questions and Answers?

Bob Whiteman

Bob.Whiteman@palmsource.com

http://www.palmos.com/dev/

http://www.palmos.com/dev/support/forums



palm OS® Cobalt

Contact: Doug MacMillan
Sr. Palm OS Platform Evangelist
Email: doug.macmillan@palmsource.com

Contest: Open Labs (Tues/Wed)
68K Compatibility & Protein Contests
Motorola sponsored Grand Prize!

MOTOROLA
digital dna